NVIDIA Omniverse Isaac Sim Environment

Januar 23, 2026

1 Omniverse Isaac Sim World Setup

1.1 Prerequisites

To set up the Omniverse Isaac Sim World, the following files are required to be downloaded.
Note: It would be helpful if you save every file in one directory/folder for easier setup.

1. Omniverse USD and fbx Files:
   (a) `MAINDEMO.usd`
   (b) `Shackleton Ridge 300x300 v4 -001.usd`
   (c) `Viper Substance File (1).fbx`
   (d) `CADREDemo.usd`
   GitHub Repositories:
   - HSML-Models
   - 3D-Models
2. Python Scripts:
   (a) `ViperAOmniConsumer.py`
   (b) `CadreAOmniAPIJSON.py`
   (c) `ViperAOmniAPIJSON.py`
   (d) `collision_HSML.py`

1.2 Setup Steps:

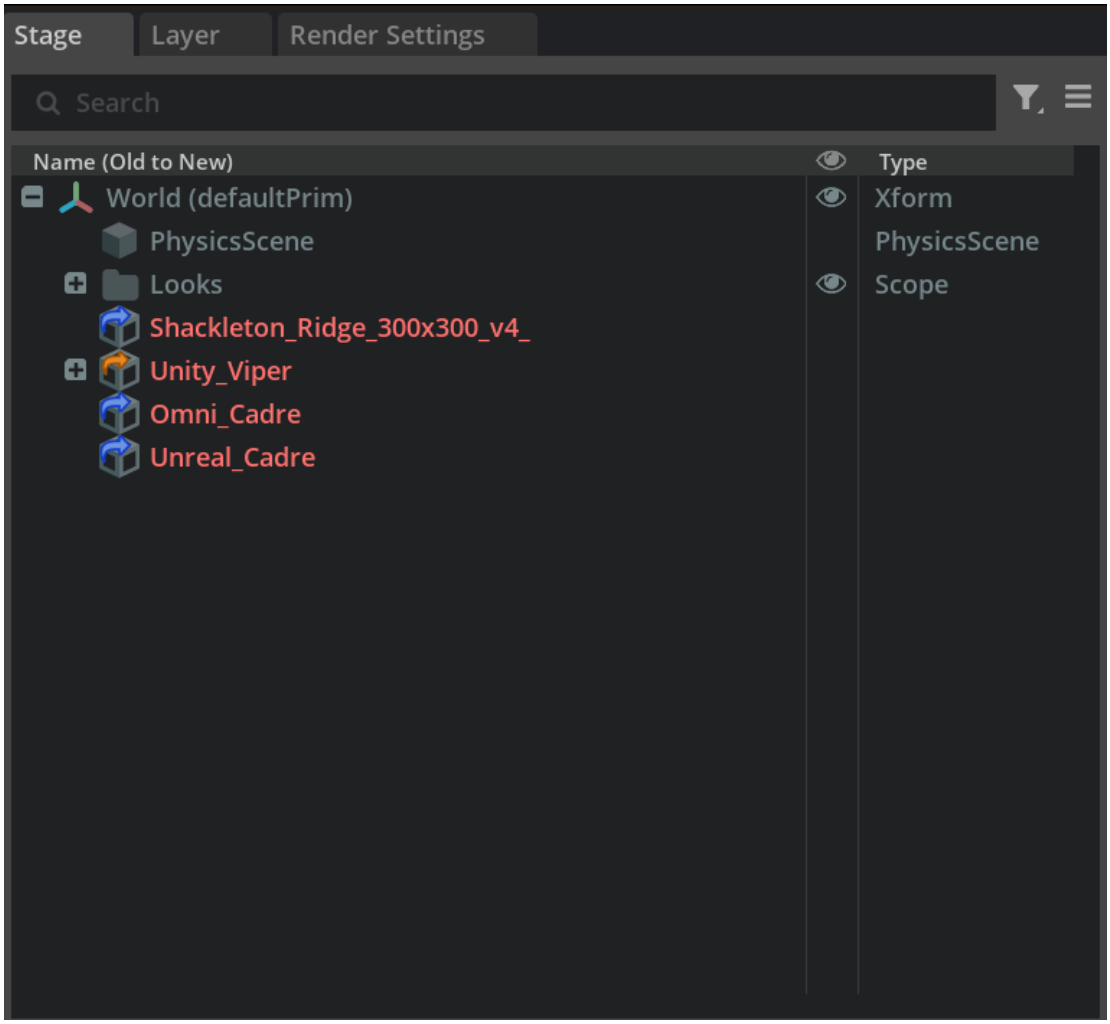
1. Open Command Prompt
2. Navigate to `isaacsim` directory/folder
   - `cd ..` to go to parent directory/folder
   - `dir` to view list of subdirectories of current directory

- `cd <dir-name>` to go into a subdirectory inside of the current directory

```
C:\Users\elija>cd ..

C:\Users>cd ..

C:\>cd isaacsim

C:\isaacsim>isaac-sim.selector.bat
```

3. Start isaac sim by executing the following command: `isaac-sim.selector.bat`
   - **Note:** for Windows Powershell, use: `.\isaac-sim.selector.bat`
4. Press 'START' on the Isaac Sim App Selector popup window
5. If a Warning pop up window for Python Scripting shows up, press "Yes"

6. Navigate to the 'Stage' window
7. The omniverse world may experience errors when loading in USD files and python scripts.
   (a) USD files that are not loaded in will be displayed with a red font

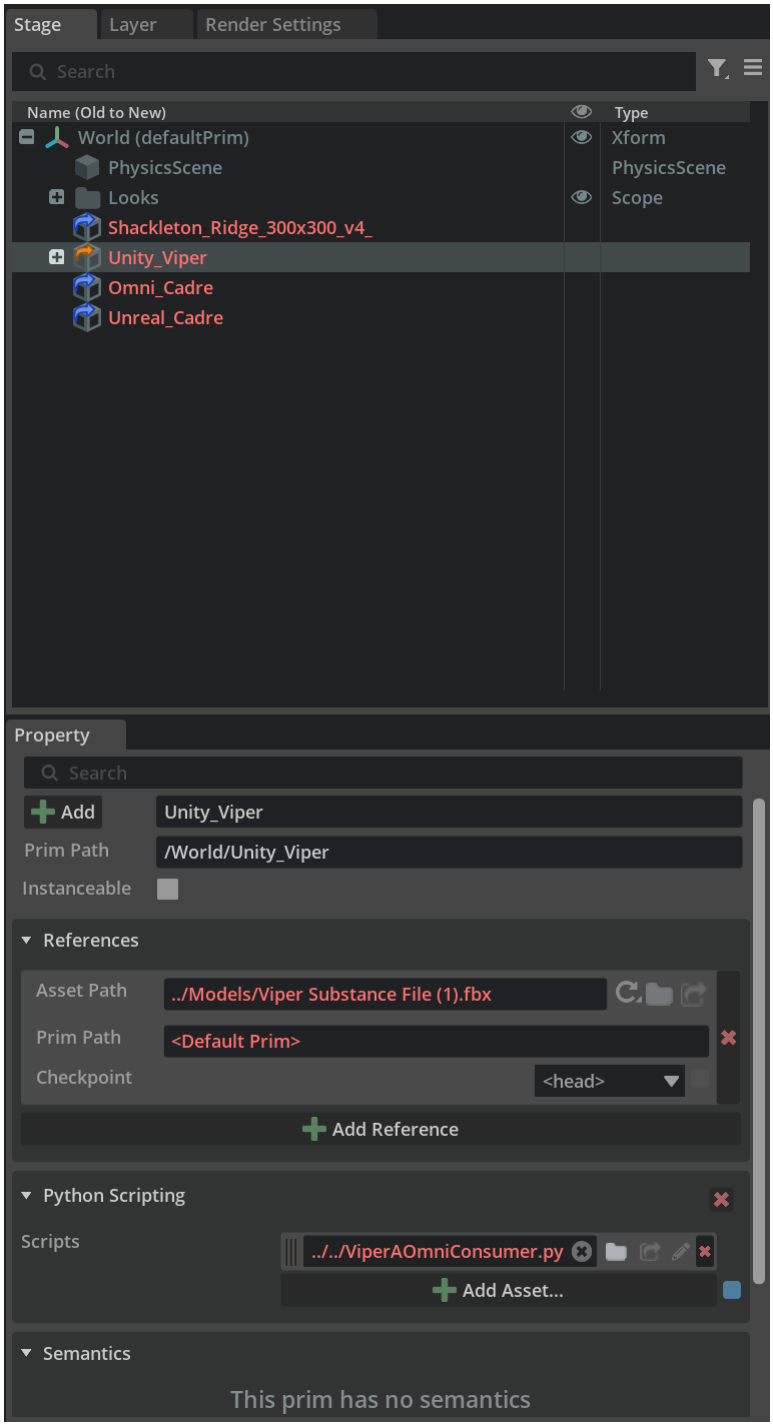


(b) To fix these errors, simply click on them

(c) For this example, we will use `Unity_Viper` as an example:

   i. Click on `Unity_Viper` on the Stage window

   ii. After clicking, its property window should pop up below it

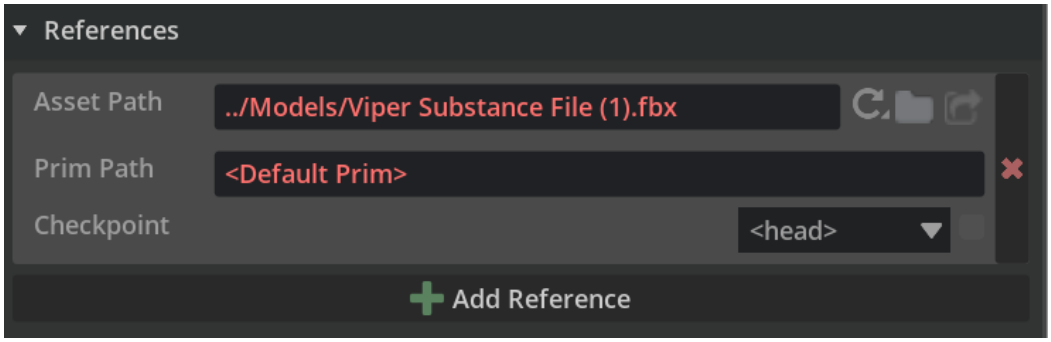


iii. The reference errors will be displayed with a red font in the property window

iv. For `Unity_Viper`, we see that there are errors under the 'References' and 'Python Scripting' properties

v. First, let's solve the 'References' error by clicking the folder icon next to the 'Asset Path'

vi. After clicking the folder, a popup window should appear asking for you to select the Reference

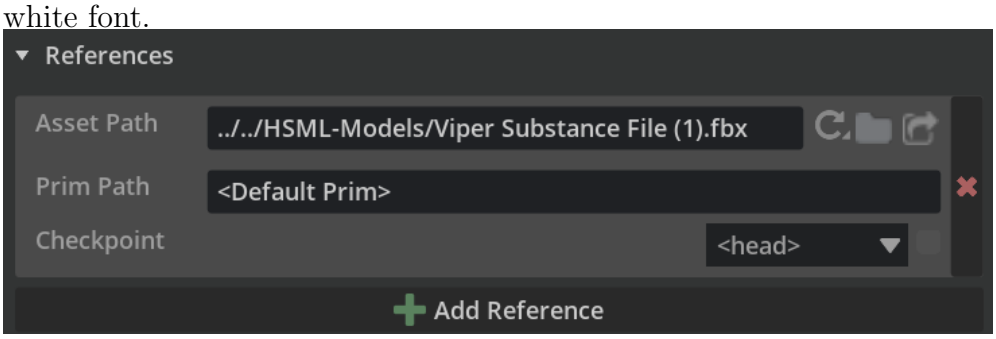


vii. Navigate to the desired file that is required to solve the error.

   A. In this case, we want to navigate to and select `Viper Substance File (1).fbx`

   B. Click on the file and then the 'Select' button

   C. This should have loaded the Asset Path, now displaying the path in a white font.



viii. Now, we need to fix the 'Python Scripting'. Similarly, we want to click on the folder icon next to the Script's path.

   A. Follow the same directions that we did for `Viper Substance File (1).fbx`

   B. After which it should load the script properly and now display a path in a white font.

8. Repeat the steps discused from 7 for the remaining reference errors in the Stage window

9. After fixing these errors, the Omniverse world should be ready to go.

2 rover_generalizer.py

Logic: I want the rovers in Omniverse to have a consistent stage tree structure and all required settings for the collision to occur.

Here, we achieve the required goals for generalizing:

1. Rigid body and revolute joints: For Omniverse to host Viper (other rover) from Unity, we need to generalize its stage tree structure and settings to allow Omniverse's physics engine to control it.

2.1 Usage

To use the `rover_generalizer.py` script, simply change the following fields in the "EXEC" portion of the code:

```
rover_root_path="/World/Viper",
output_parent="/World/Viper",
rover_name="Viper",
```

Other fields like the scale factor, wheel velocity, and drive damping can be adjusted accordingly by users.

After editing the fields above, simply run the python script inside Omniverse Isaac Sim.

3 collision_HSML.py

Logic: I want the rover that is controlled by Omniverse to switch from producer to consumer after a collision happens. The switch needs to happen automatically and based on detecting collisions between two rigid body colliders.

Here, we achieve the required goals for collision:

1. Collision detection: the rigid body prims of each rover is tracked and once they intersect or meet, the Omniverse rover (Cadre) gets turned into a kinematic rigid body, disabling physics from Omniverse.

2. Producer-Consumer switch: a hosting flag field is kept to see the required set up of which rover is to switch from a producer to consumer and vice versa.

3.1 Setup

To provide ease of setup, `collision_HSML.py` includes a CONFIG dictionary located at the top for all of the set up required. This section provides an overview of each element in the dictionary.

1. `"prim_paths"`: Refers to various prim paths in the Omniverse world that are required for the script to run. These paths identify the prims of the primary rover to which the script is attached and the other rover that is in the world

- "self_chassis": Refers to the chassis prim to which the script is attached to
- "other_rover_chassis": Refers to the chassis prim of the other rover that is in the world
- "wheel_joints": Refers to the wheel joints of the chassis prim to which the script is attached to. For Cadre and Viper, there should be an array with 4 values

2. "drive": Refers to the Joint Drive configuration. These are usually not touched. One value that can be changed in this sub-dictionary is "default_velocity"

3. "io": Refers to Producer and Consumer json file paths.

   - "producer_output": contains various fields that allow users to define a definite output file path or allow the usage of the tmp directory.
   - "consumer_input_path": Refers to the file path of the JSON to read in for when the switch happens.

4. "consumer_schema": Refers to the expected schema of the consumer JSON file.

5. "collision": contains various fields that help with the collision detection.

6. "behavior": contains various fields that controls the role of Omniverse

   - "hosting_default": True if Omniverse is hosting, False if it is not.
   - "start_tracking_default": True if Omniverse is controlling the rover, to which the script is attached to, originally.
   - "switch_to_coonsumer_on_collision": True if Omniverse is switching off to consuming if a collision happens. Must be the opposite value of "hosting_default"

7. "colliders": Allows users to provide colliders. Providing colliders is not necessary, as they are discovered automatically.

8. "export": controls which Physics parameters and information is included into the JSON file that is sent.